ZERO TRACE — UNIFIED SOLUTION

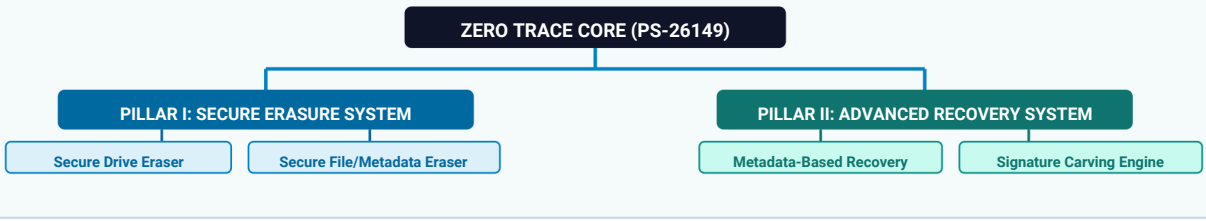
INTEGRATED STORAGE SANITIZATION & ADVANCED FILE RECOVERY ECOSYSTEM

Problem Statement: PS-26149

Grand Finale Architecture • Technical Repor t

Executive Summary: The Zero Trace framework fulfills the dual mandates of **PS-26149** by synthesizing verified, tamper-evident media sanitization with dual-path forensic recovery into a single coordinated ecosystem. While enterprise security requires verifiable, unrecoverable data destruction aligned with NIST SP 800-88 guidelines, investigative response requires non-destructive, adaptive data salvaging across both intact directory structures and damaged, corrupted physical disk blocks.

1. HIGH-LEVEL ARCHITECTURAL DIVISION



2. UNIFIED TECHNOLOGY STACK & INFRASTRUCTURE

Erasure Engine & Desktop UI

RUST • TAURI V2

- **Memory-Safe Kernel Core:** Implemented in native Rust for determinism, direct IOCTL hardware binding, and zero garbage-collection latency.
- **Application Shell:** Tauri v2 binding with React.js & Tailwind CSS desktop client for minimal memory overhead compared to Electron.
- **Audit Trail:** SQLite structured store coupled with RocksDB hash-chained ledger and SHA-256 state tracking.
- **Isolated Environment:** QEMU virtualized disks for non-destructive VM testing.

Forensic Recovery Core

PYTHON • PYTSK3

- **Filesystem Metadata Analysis:** Python wrapped with `pytsk3` (The Sleuth Kit C bindings) for raw unallocated MFT/FAT record parsing.
- **Advanced Carving Pipeline:** Standalone block streaming scanner using binary byte signatures and Pillow image decoding integrity checks.
- **Interoperability:** Command-line execution, structured JSON outputs, and Tkinter asynchronous threading for live progress dispatch.
- **Test Harnesses:** Pytest automated suite and synthetic image injectors.

One-Line Definitive Architecture Description

"Zero Trace is an integrated, safety-first storage sanitization and dual-path forensic recovery ecosystem combining hardware-aware, cryptographically audited sanitization (Rust) with metadata reconstruction and raw byte carving (Python/pytsk3) to deliver complete, auditable lifecycle data governance."

3. PILLAR I: SECURE ERASURE SYSTEM (FORENX CORE)

The erasure subsystem guarantees that sensitive data is irrevocably expunged while actively preventing collateral destruction of host systems through strict fail-closed safety semantics, capability-based method resolution, and cryptographic auditability.

A. Secure Drive Eraser Pipeline



Device Classification & Capabilities

Identifies physical storage targets via Windows Storage APIs & low-level IOCTL. Resolves bus protocols:

- **ATA / SATA:** Security Erase Unit, Enhanced Secure Erase.
- **NVMe:** Format NVM (Cryptographic Erase vs. User Data Erase).
- **SCSI / SAS:** Sanitize (Block Erase, Overwrite, Crypto Scramble).
- **States:** Evaluated strictly as *Supported*, *Unsupported*, or *Unknown*.

Storage Technology Adaptations

Differentiates rotational and solid-state hardware dynamics:

- **HDDs:** Host sequential overwrite patterns destroy physical magnetic track domain states reliably.
- **SSDs / NVMe:** Wear-leveling and Flash Translation Layer (FTL) over-provisioned blocks bypass host logical overwrites. Enforces hardware-level Sanitize / Crypto Erase.

B. Secure File & Metadata Eraser Pipeline



Target & Directory Traversal Protection

- **Path Normalization:** Resolves canonical paths to verify target authenticity and prevent relative traversal jailbreaks.
- **Reparse-Point & Junction Shield:** Prohibits traversal through NTFS symbolic links or volume junctions, protecting host drives from inadvertent erasure loops.
- **Filesystem Probing:** Analyzes cluster geometry and directory record attributes on NTFS, FAT32, and ext-family mounts.

Residual Area Sanitization Model

- **Content Eradication:** Multipass targeted sector overwriting.
- **Metadata Scrambling:** File names, attributes, and file system timestamps randomized prior to unlinking.
- **Alternate Data Streams (ADS):** Enumerates and truncates NTFS hidden fork data streams.
- **Slack & Free Space:** Cleans cluster slack to eliminate partial-block residual leakage.

Safety Invariants & Cryptographic Audit Ledger

FAIL-CLOSED SECURITY

System/Boot Partition Shield: Hard-coded safetyinterlocks intercept and reject sanitization attempts targeted at active OS installations or system EFI boot structures. If hardware capability responses are ambiguous, the engine defaults closed.
Tamper-Evident Ledger: Every operation calculates `SHA-256(Prev_Hash || Current_Event_Canonical_Data)`, producing an immutable cryptographic audit record stored in SQLite/RocksDB for audit defense.

4. PILLAR II: ADVANCED RECOVERY SYSTEM (DUAL ARCHITECTURE)

Zero Trace recognizes that data loss manifests in two distinct forensic states. Rather than relying on a single fallible recovery mechanism, it couples metadata tree walking with deep signature carving.

A. Metadata-Based File & Folder Recovery (Intact Records)



Operational Mechanics: Utilizes `pytsk3` bindings to parse filesystem structures directly without mounting. Traverses FAT16 directory tables and ext4 inode trees to locate deleted entries flagged as unallocated. Crucially, it validates data block allocation status before reconstruction—if underlying clusters are intact, it recovers the file byte-for-byte while **preserving original filenames, extensions, and directory paths**.

B. Signature-Based Drive Recovery (Missing / Corrupted Records)



Operational Mechanics: Operates entirely independent of filesystem records (post-format or partition wipe). Streams binary data in sliding buffers searching for file markers (e.g., JPEG Start-of-Image `FF D8` to End-of-Image `FF D9`). Carves discovered spans, applies reconstruction to damaged headers, validates image decodability using Pillow (PIL), and assigns a confidence metric (`0.000–1.000`) categorizing output as *Reconstructed*, *Partial*, or *Rejected*.

Comparative Forensic Evaluation: Metadata Recovery vs. File Carving

Feature Dimension	Metadata-Based Module (pytsk3)	Signature Carving Module (PIL Engine)
Operational Target	Filesystem metadata records (MFT / FAT entries / Inodes)	Raw sequential byte streams on unallocated disk space
Prerequisite State	Filesystem table must be partially intact	Zero filesystem requirements (damaged / formatted disks)
Metadata Preservation	**High:** Restores original filenames, paths, & dates	**None:** Assigns sequential IDs (e.g., `REC-0001.jpg`)
Fragmentation Handling	Follows filesystem block allocation maps cleanly	Linear extraction; requires heuristic reconstruction
Source Image Integrity	Guaranteed strictly read-only; no disk writes	Strictly read-only binary chunked streaming

5. TEST EXECUTION & EMPIRICAL VERIFICATION RESULTS

Module Name	Test Harness & Target	Execution Metric	Verified Operational Status
ForenX Secure Eraser Core	Cargo Test & Check (`--workspace`)	**202 passed**, 0 failed	Safety models, IOCTL parsing, fail-closed interlocks validated
Drive Recovery (Carver)	Pytest Framework (`pytest -q`)	**67 passed**, 0 failed	JPEG validation, carving CLI, decoding error boundaries verified
File & Folder Recovery	Synthetic Images (`make_test_image.py`)	**100% Hash Match**	FAT16 & ext4 byte-for-byte identity; nested folder paths preserved

6. CURRENT MVP SCOPE VS. PLANNED FUTURE ROADMAP

Component	Implemented MVP Scope (Evaluated)	Future Enterprise Roadmap
Sanitization Engine	Target discovery, capability modeling, mock execution, RocksDB hash-chain ledger, Tauri GUI.	Kernel-mode destructive purge drivers, physical NVMe Sanitize execution, MinIO remote audit vault.
Metadata Recovery	FAT16 & ext4 direct metadata tree parsing, deleted flag extraction, byte-for-byte cluster extraction.	NTFS MFT parser, exFAT structures, live folder restoration directly into explorer, pytest suite.
Carving Engine	Chunked raw image reader, JPEG SOI/EOI carving, Pillow integrity decoding checks, Tkinter GUI.	Expanded signatures (PNG, PDF, MP4, MP3), deep fragmented cluster reassembly, AI-assisted carving.

Project Repositories & Module Integration

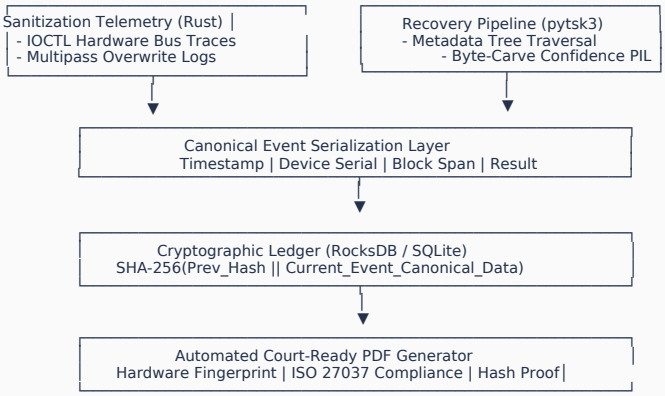
PS-26149 COMPLIANCE

The Zero Trace architecture maintains modular repositories across its functional pillars, enabling independent verification of each specialized engine while maintaining unified IPC and CLI interfaces for full-suite integration:

- **ForenX Secure Eraser:** `crates/drive_eraser`, `crates/file_eraser`, and `src-tauri` desktop shell.
- **Drive Recovery (Carver Engine):** `github.com/AhanKamal/zero-trace-drive-recovery` (Pillow validator, Pytest suite).
- **File & Folder Recovery (Metadata Engine):** `github.com/inchara737/sih-recovery-module` (`pytsk3` filesystem reader, synthetic FAT16/ext4 image generator).

7. FORENSIC REPORTING & AUDIT VERIFICATION ENGINE

The Zero Trace reporting subsystemtranslatesdeep recovery records andlow-level sanitization events into an automated, tamper-evident forensic artifact suitable for statutory compliance and courtroom admissibility. Rather than generating static text logs, the engine links hardware-level telemetry, file system reconstruction traces, and cryptographic states into an automated, self-verifying report.



A. AUTOMATED EVIDENTIARY REPORTING ARCHITECTURE (COURT-READY PDF)

- Metadata & Audit Synthesis: The generator aggregates low-level extraction and eradication records stored in SQLite with immutable audit entries maintained in RocksDB.
- Reconstruction Telemetry: For each salvaged artifact, the report itemizes the source block addresses, recovery mechanism used (pytsk3 metadata walk vs. binary signature carving), file size, MIME type, and Pillow decoding confidence score (0.000–1.000).
- Statutory Compliance & Legal Guardrails: Reports are formatted to adhere to strict digital evidence standards (ISO/IEC 27037, NIST SP 800-88, and statutory evidence admissibility frameworks such as Section 63 of the Bharatiya Sakshya Adhiniyam, BSA 2023 / Indian Evidence Act Section 65B):
- Drive serial numbers, bus interface parameters (ATA/NVMe/SCSI), and physical geometry.
- Operator identity, execution duration, and UTC microsecond timestamps.
- SHA-256 source media pre-acquisition and post-acquisition integrity digests.
- Automated non-destructive read-only assertion declarations.

B. CRYPTOGRAPHIC AUDIT LEDGER & STATE BLUEPRINT

To prevent retrospective alteration of activity logs by operators or compromised host systems, Zero Trace records all forensic actions into a mathematically linked ledger.

```
Current_Hash = SHA-256(Prev_Hash ||
Current_Event_Canonical_Data)
```

- Chained Hashing Primitive: Every runtime operation computes a canonical state digest using the formula above.
- Immutability Invariant: The resulting hash chain is stored across an append-only RocksDB instance coupled with SQLite structured indexing.
- Audit Defense & Verification: Any tampering, omission, or truncation of historical logs breaks the hash-chain link downstream, alerting examiners and opposing counsel during judicial verification.

C. COMPARATIVE OVERVIEW: LOG CAPTURE VS. EVIDENTIARY PDF EXPORT

Dimension	Raw Audit Ledger (RocksDB / SQLite)	Automated Court-Ready PDF Report
Primary Consumer	Internal engine verification, automated audit crawlers	Judicial authorities, forensic examiners, compliance officers
Data Format	Key-value records, canonical JSON byte strings	Standardized, non-editable PDF document
Integrity Model	Continuous SHA-256 chained hash ledger	Embedded root hash, digital signing block, hardware fingerprint
Content Scope	Micro-operations (IOCTL writes, block offsets, stream reads)	High-level findings, recovery metrics, integrity proofs, device specs
Retention Mode	Local fast append-only store with local persistence	Archival export, court-submission ready, air-gap deliverable

D. FUTURE FORENSIC & REPORTING ROADMAP

- MinIO Remote Audit Vault: Off-site synchronization of encrypted audit ledgers across air-gapped physical enclaves.
- Live Shell & MFT Integration: Expansion of pytsk3 parsing directly into live Windows Explorer shells alongside full NTFS Master File Table ($MFT) and exFAT mapping.
- Expanded Signatures & AI Reassembly: Inclusion of advanced headers (PNG, PDF, MP4, MP3) and AI-assisted deep fragmented cluster reassembly for non-contiguous sectors.
- Cryptographic PDF Attestation: Native X.509 PKI hardware-token digital signatures embedded directly within the generated PDF document.

8. ZERO TRACE NEXT-GEN CORE

ZERO TRACE NEXT-GEN CORE

ROADMAP ADVANCEMENTS	NOVEL DIFFERENTIATORS
• AI Neural File Carving	• Blockchain-Anchored Audit Root
• Direct Kernel Purge Engine	• HPA / DCO Covert Storage Hunter
• Remote Air-Gap Audit Sync	• Journal-Aware Micro-Eradication

1. AI-DRIVEN NEURAL FILE CARVING & FRAGMENTED CLUSTER REASSEMBLY

- What it does:
  Signature carvers extract contiguous bytes between start-of-file and end-of-file markers. When a drive is fragmented, recovered files end up split, corrupted, or unreadable. This engine predicts cluster relationships using deep learning to piece together fragmented chunks without filesystem allocation tables.
- Technologies to use:
- Model: Lightweight 1D-CNN or Transformer-based sequence classification model trained on file-fragment transitions.
- Runtime: ONNX Runtime integrated into Rust via ort bindings for low-latency, CPU inference.
- Validation: Multi-format integrity parsers using Pillow (images), PyMuPDF, and ffmpeg/ffprobe.
- Why it stands out: Commercial forensic suites struggle with fragmented raw files once filesystem tables are destroyed. Neural reassembly turns partial, corrupted evidence into readable files.

2. KERNEL-MODE HARDWARE PURGE ENGINE (DIRECT NVME & ATA CONTROLLER PASS-THROUGH)

- What it does: Replaces OS-level IOCTL calls with a custom signed kernel driver that communicates directly with the storage controller firmware. It triggers hardware-level crypto-scramble and block-erase commands (NIST SP 800-88 "Purge" and IEEE 2883 "Sanitize").
  Technologies to use:
- Language & Framework: Rust Windows Driver Framework (windows-drivers-rs / KMDF).
- Protocols: Direct pass-through execution of NVMe Admin Commands and ATA command pass-through.
- Why it stands out: Solid-state drives utilize Wear-Leveling and Flash Translation Layers, meaning logical overwrites miss reserved over-provisioned blocks. A kernel-level pass-through issues the command directly to the disk controller, sanitizing every flash cell within milliseconds.

3. AIR-GAPPED REMOTE AUDIT VAULT SYNCHRONIZATION

- What it does: Allows field investigators using air-gapped forensic laptops to securely export, batch, and cryptographically synchronize RocksDB audit trails to an off-site centralized repository upon reconnecting to an isolated network.
- Technologies to use:
- Storage Backend: MinIO S3 Object Storage deployed on private government enclaves.
- Cryptographic Transport: Mutual TLS (mTLS) with pinned X.509 client certificates and hardware-backed signing via YubiKey / PKCS#11 tokens.
- Why it stands out: Local logs can be physically confiscated or disputed in court. Automated sync ensures enterprise audit defense without breaking air-gap field protocols.

4. BLOCKCHAIN-ANCHORED PROOF OF SANITIZATION & CHAIN OF CUSTODY

- What it does: Calculates the Merkle Root of an entire sanitization/recovery session and commits that root hash into a smart contract on a permissioned or public ledger.
- Technologies to use:
- Merkle Engine: Rust merkle-tree crate calculating a rolling Merkle tree of every block operation and audit event.
- Web3 Integration: ethers-rs or alloy-rs compiled directly into the core.
- Smart Contract: Solidity contract logging the Drive Serial Hash, Merkle Root, Timestamp, and Operator Public Key.
- Why it stands out: Industry-standard tools issue proprietary PDF certificates that can be edited to fake an audit. A decentralized, on-chain Merkle proof provides mathematical, third-party verifiable proof that data was destroyed at a specific second.

8. ZERO TRACE NEXT-GEN CORE (CONTINUED)

The following modules extend the platform from conventional recovery and sanitization into covert-storage discovery, surgical metadata erasure, and volatile-memory acquisition.

5. COVERT STORAGE CHANNEL HUNTER (HPA, DCO, AND SLACK-SPACE SCANNER)

- What it does: Threat actors and malware hide files inside physical drive zones that the operating system cannot see: Host Protected Areas (HPA), Device Configuration Overlays (DCO), and unallocated volume slack. This module unmasks these hidden sectors before running recovery or sanitization.
- Technologies to use:
- Low-Level ATA/SCSI Probing: Issuing READ NATIVE MAX ADDRESS vs. IDENTIFY DEVICE sector comparisons to detect capacity differences.
- NTFS Parsing: Direct extraction of Alternate Data Streams and cluster-slack scrapers built in Rust.
- Why it stands out: Standard forensic recovery software only parses visible partition tables. If an attacker hid 10GB of exfiltrated data inside an HPA, standard tools report the drive as clean. Zero Trace identifies and extracts covert storage structures.

6. JOURNAL-AWARE SURGICAL MICRO-SANITIZATION

- What it does: Traditional shredders overwrite the contents of a target file, but they leave filenames, timestamps, and cached copies in the NTFS $LogFile, $UsnJrnl, $MFT, and Volume Shadow Copies (VSS). The surgical eraser sanitizes the target file and hunts down every operational trace of that file across the file system metadata without corrupting adjacent files.
- Technologies to use:
- NTFS Metadata Parser: Direct raw binary modification of the $MFT record.
- Shadow Copy Cleaner: Windows VSS COM APIs to purge related snapshot differential fragments.
- Free Space Scrubbing: Targeted overwrite of the specific cluster slack occupied by the targeted file.
- Why it stands out: Existing tools either wipe the whole disk or leave forensic breadcrumbs across system journals. Surgical micro-sanitization provides true "anti-forensic clean erasure" for legal redaction cases where an agency needs to destroy specific files while keeping others intact.

7. VOLATILE MEMORY (RAM) LIVE-CAPTURE & EPHEMERAL KEY PRESERVATION

- What it does: Captures volatile RAM into an AFF4 forensic container in one click to preserve BitLocker encryption keys, mounted container credentials, and active malware processes before wiping or recovering a disk.
- Technologies to use:
- Driver: WinPmem or raw \Device\PhysicalMemory ring-0 handle acquisition.
- Analysis Bridge: Volatility 3 core integrated via Python backend to scan for AES keys and active network sockets.
- Output Container: AFF4 stream with SHA-256 block hashing.
- Why it stands out: Disk forensic software and memory capture utilities are typically sold as separate tools. Unifying volatile and non-volatile acquisition into a single application gives first responders an all-in-one incident response solution.